TAKE-HOME SUBMISSION · CONVERSATION ANALYSIS BOT FOR CONTACT CENTRE INSIGHTS

Conversation Intelligence Copilot

How I designed, built and evaluated an evidence-first analytics agent for 3,000 customer-support conversations, with the language model optional.

4 / 4 assignment questions answered and independently re-computed	**95.9%** intent routing on 73 held-out questions (from 61.6%)	**125 ms** median answer time on the deterministic path	**$0** to run: no API key; the optional model is local

This report covers the problem framing, dataset handling, system design, agent flow, model and tool choices, the three bonus extensions, evaluation, limitations and next improvements, as the brief requests. It is written in the first person because most of what matters here is judgement: what I chose to build, what I chose *not* to build, and why.

Implementation repository: `github.com/florina787/Data-sets` (folder `conversation-intelligence-copilot`, branch `claude/happy-edison-gef7ay`). A ZIP can be built from it with `make package`. The next pages map every item in §6 of the brief to where it is answered.

Prepared by Florina · October 2026

https://github.com/florina787/Data-sets/tree/claude/happy-edison-gef7ay/conversation-intelligence-copilot

CONTENTS

What is in this report

Submission checklist · How this submission meets §6 of the brief . . . 3

Summary · What I built, in one page . . . 4

1. Problem framing · Reading the brief as an analyst would . . . 4

2. Dataset handling · Preparing 3,000 conversations, and what the data hid . . . 5

3. System design · Components, contracts and the data passed between them . . . 6

4. Agent flow · From question to checked answer . . . 8

5. Model and tool choices · What I chose, what I rejected, and what it costs . . . 10

6. Results · The four questions, answered . . . 11

7. Bonus extensions · Retrieval, a specialised agent, and memory . . . 13

8. Beyond the brief · Six enhancements . . . 15

9. Evaluation · What I measured, what worked, and what failed . . . 16

10. Judgement · What I did, what I chose not to do, and how I worked . . . 18

11. Limitations · What a reviewer should know . . . 18

12. Next improvements · From prototype to production . . . 19

13. Repository · What is in the repository, and how to run it . . . 19

SUBMISSION CHECKLIST

How this submission meets §6 of the brief

The brief asks for a short report and the implementation repository. This page maps each requirement to where it is answered, so nothing has to be searched for.

The report covers

REQUIRED TOPIC	WHERE IN THIS REPORT
Problem framing	§1 Problem framing
Dataset handling	§2 Dataset handling: preparation steps, assumptions, derived fields, labels, aggregations
System design	§3 System design: components, contracts, schemas and data passed between agents
Agent flow	§4 Agent flow: the four-stage graph, one request end to end, routing, failure handling
Model and tool choices	§5 Model and tool choices: alternatives, dependencies, API keys, cost assumptions, and why a local model instead of a hosted API
Evaluation: representative successes and failures	§9 Evaluation: Representative successes; Representative failures
Evaluation: task-specific metrics	§9 Evaluation: Task-specific metrics
Evaluation: grounding quality	§9 Evaluation: Grounding quality
Evaluation: latency or consistency	§9 Evaluation: Latency and consistency
Limitations	§11 Limitations
Next improvements	§12 Next improvements

The repository includes

REQUIRED ITEM	WHERE IN THE REPOSITORY
Source code and clear setup/run instructions	`backend/`, `frontend/`; README · Quick start (`make install / prepare / index / dev`), IDE setup, Docker
Sample requests and outputs	`examples/requests/` (6 requests) and `examples/outputs/` (JSON + readable Markdown), generated from real runs
Instructions for obtaining or preparing the dataset	Supplied file committed at `data/raw/cs_conversations.csv.zip`; `make prepare`; `docs/dataset/DATASET.md`
Environment-variable requirements and external-service notes	`.env.example`; README · Configuration and Models, dependencies and cost. No API key or external service is required.

Details for each repository item are in §13.

SUMMARY

What I built, in one page

Contact-centre leaders make decisions from conversation analytics. If a number in that analysis is wrong, people act on it anyway. So my first design decision came before any code: **numbers are computed, never generated.** Every count, rate and rank in an answer comes from a registered, parameterised DuckDB operation. A language model may help interpret a question or phrase an explanation, but it never produces a figure and never writes SQL.

On top of that rule I built an analyst-facing copilot: a React workspace and a FastAPI service around a LangGraph workflow of specialised agents. An analyst asks a question in plain language. The workflow then:

- decides whether a model is needed at all (a **Zero-LLM router**);
- plans which registered operations answer the question;
- computes the result and retrieves the source transcripts that illustrate it (hybrid FAISS search);
- explains the result, choosing charts that fit its shape;
- audits the answer with a QA agent and an **Evidence Challenger** before it is shown.

The answer carries counts and rankings, charts, cited conversations with message IDs, uncertainty notes, and a replay of every step. All four example questions in the brief are answered with zero model calls.

WHAT I DID	WHAT I DELIBERATELY DID NOT DO	HOW
Answered all four questions end to end, each independently re-computed from the raw CSV	Rank individual agents for coaching: no agent has more than two conversations	A typed agent state, a registry of 15 DuckDB operations, one versioned result schema
Built all three bonus extensions: hybrid retrieval, a local LLM intent agent, session memory	Let a model write SQL, produce numbers, or claim causes	Numeric grounding check on every explanation; an adversarial Evidence Challenger
Measured everything: routing on a frozen held-out split, retrieval, QA, latency, 163 automated tests	Tune on the test split, or report the flattering retrieval number after the data changed	Benchmarks written before changes, frozen by SHA-256, scored once
Made it run with no API key and no cost	Depend on a hosted model, or append new data	A local Qwen2.5-1.5B model on CPU, used only when the rules are unsure

1 · PROBLEM FRAMING

Reading the brief as an analyst would

The brief asks for a prototype that helps analysts *make better decisions*, and for answers that are clear and evidence-based. I read that as three requirements, in this order of importance:

- **Correct.** A plausible wrong answer is the worst outcome for a decision-support tool, because it is trusted.
- **Checkable.** An analyst must be able to see where a number came from and read the conversations behind it.
- **Honest about uncertainty.** When the data cannot support a conclusion, the tool should say so rather than produce one anyway.

> MY THINKING
>
> *The obvious build is one prompt: put the transcripts in front of a large model and ask the question. I rejected it early. It cannot be tested properly, its counts drift between runs, and it hides which data it used. The brief also asks explicitly for a deliberate workflow rather than a single prompt. I wanted the opposite extreme: a system whose facts are deterministic and testable, with a model added only where interpretation genuinely helps.*

Who the user is. A contact-centre analyst or team lead who knows the business but not the data model. They ask questions like the four in the brief, follow up ("compare the top two", "what about the previous month?"), and need something they can paste into a leadership update without re-checking every figure.

What success meant. Each of the four questions answered correctly and verifiably. Every number traceable to an operation. Every claim backed by retrievable evidence. Refusals for questions the data cannot answer. And a system that a reviewer could run in minutes, with no key.

2 · DATASET HANDLING

Preparing 3,000 conversations, and what the data hid

I used only the supplied sample of the Syncora.ai Customer Support Conversation dataset (Kaggle): 3,000 synthetic conversations, 41,965 messages, from 1 August to 2 December 2025. **No conversations were appended.** I generated 120 tagged synthetic conversations for UI demos only; they are excluded from every default build and analysis.

Preparation steps

- **Ingest and validate:** schema checks, duplicate and null checks, per-column profiles (data/manifests/).
- **Clean:** every message ends in random filler tokens. I stripped them into `clean_text` and kept the raw text.
- **Mask:** PII patterns are masked into `masked_text`, which is the only text sent to any model.
- **Order:** timestamps are not in turn order in any of the 3,000 conversations, so the turn index is authoritative and each conversation is dated by its earliest message.
- **Label:** after cleaning there are only 426 distinct message templates. I labelled the templates once with a documented rule set (act, sentiment score, empathy, courtesy, resolution fit, topic, product) instead of labelling 41,965 messages.
- **Store:** DuckDB tables `conversations`, `messages`, `message_labels` and `topics`, with build metadata.

Derived fields, labels and aggregations

FIELD	DEFINITION
Conversation sentiment	Mean of customer-turn sentiment scores, rounded to 9 decimals; **negative** when ≤ −0.25
Primary topic	Topic of the customer's opening message (each conversation is counted once)
Conversation month	Month of the earliest message; "last month" = latest month in the data (November 2025)
Acknowledgement rate	Share of negative customer turns whose next agent turn acknowledges or apologises
Closing-action fit	Whether the agent's closing action matches the issue (documented mapping; general help excluded)
Escalation / urgency	Customer turns explicitly asking to escalate, or pressing for urgency
Priority score (Q4)	Weighted min-max signals: volume 0.30, negative rate 0.25, escalations 0.15, unresolved 0.15, misfit 0.10, month-on-month change 0.05

MY THINKING

The 426 templates looked like a problem at first, because the data is obviously generated. I decided to treat them as an opportunity instead. A documented template mapping is transparent and exactly reproducible, and every reviewer can disagree with a specific label. The cost is that the labels are my judgement. I measured agreement with an independent lexicon (78% accuracy, Cohen's κ 0.67), built an in-app panel to collect human labels, and I say in the evaluation that human agreement has not been measured yet.

Every interpretation (windows, thresholds, mappings) is written down in ASSUMPTIONS.md, and the data dictionary is in docs/dataset/DATASET.md.

3 · SYSTEM DESIGN

Components, contracts and the data passed between them

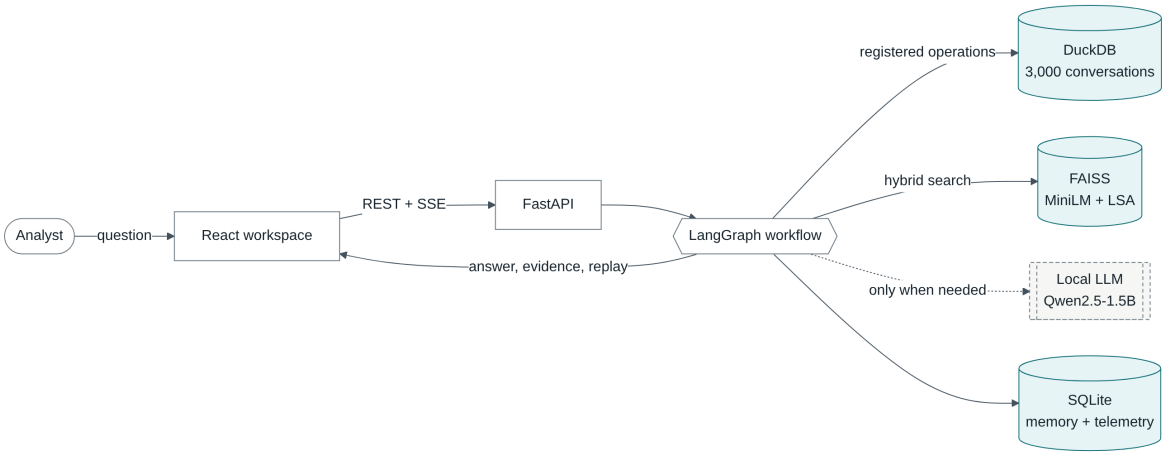


Figure 1. System overview. The model (dashed) is optional and local.

COMPONENT	RESPONSIBILITY	NEVER DOES
Zero-LLM router (`agents/router.py`)	Decides whether a model is needed; records why	Call a model
Intent router (`agents/intent_router.py`)	Turns a question into a typed intent	Invent IDs, numbers or SQL
Supervisor (`agents/supervisor.py`)	Resolves the window and references; picks operations	Compute numbers
Specialists + query registry (`analytics/*`)	Run 15 registered, parameterised DuckDB operations	Accept free-form SQL
Retrieval (`retrieval/index.py`)	Hybrid FAISS search for illustrative transcripts	Produce numbers
QA + Evidence Challenger	Verify quotes, samples, wording and grounding before display	Change a number
Memory + telemetry (SQLite)	Store references for follow-ups; record every step for replay	Reuse model prose as fact

Contracts

Agents share one typed state (`agents/state.py`). Every analysis returns the same versioned `AnalysisResult`: parameters, scope, metrics, a table, chart series, evidence references, method notes, uncertainty notes and coverage. The API wraps results in a `ChatResponse`: headline, answer, KPIs, charts, evidence, recommendations, QA results, routing decision and a replay ID. Because every operation speaks the same schema, the QA checks, charts and memory work for any analysis without special cases.

> **MY THINKING**
>
> *I chose a registry of named operations over letting a model write SQL. It costs flexibility: a question with no matching operation is answered with a clarification, not an improvised query. In exchange, every operation is unit-tested against hand-counted fixtures, and the set of things the system can claim is finite and reviewable. For a prototype that leaders might quote, that was the right trade.*

4 · AGENT FLOW

From question to checked answer

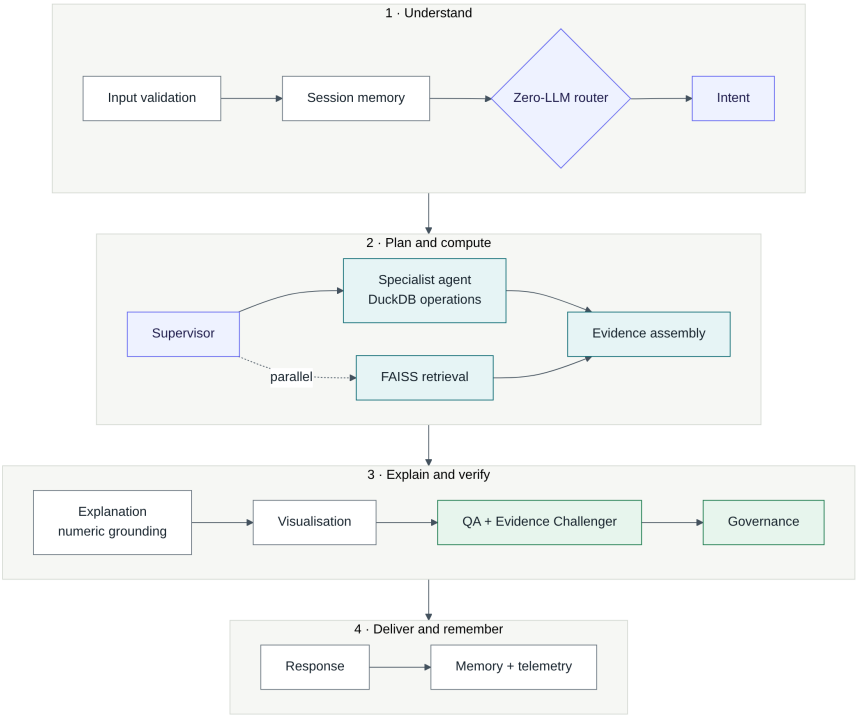


Figure 2. The LangGraph workflow in four stages. Every box is a real node with start and finish events.

A request, end to end

- **Input validation** enforces length limits and scans for prompt injection. **Session memory** loads the previous turn's intent, window and ranked keys.
- The **Zero-LLM router** and **intent** stages decide what is being asked (Figure 3). For "Which recurring concerns should leadership review first?" a high-precision rule matches, so the route is *analytics only*.
- The **supervisor** plans `leadership_priorities`, `sentiment_trend` and `resolution_quality`. The **specialist** runs them; **retrieval** fetches three negative transcripts for the top concern.
- **Explanation** writes the narrative (deterministic, or model-written and then checked for unverified numbers). **Visualisation** picks charts and says why.
- **QA and the Evidence Challenger** re-read every quoted excerpt from the database (34 of 34 matched in this example), check denominators and sample sizes, and state counter-evidence. **Governance** adds review flags.
- The **response** is returned with a replay; **memory** stores the ranked keys for follow-ups.

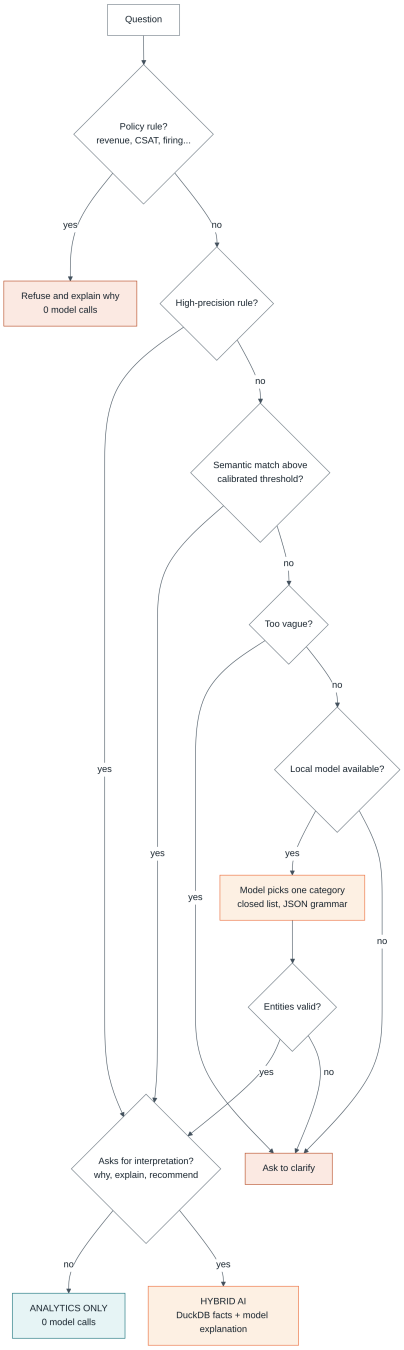


Figure 3. The routing cascade. Policy refusals always win; the model is the last resort, not the first.

MY THINKING

Failure handling follows the same principle. If the model is unavailable, slow or returns something invalid, the request falls back to the deterministic path rather than failing; a circuit breaker stops repeated calls. If an explanation names a number that is not in the verified result, it is discarded. The worst case is a plainer answer, never a wrong one.

5 · MODEL AND TOOL CHOICES

What I chose, what I rejected, and what it costs

CHOICE	WHY	ALTERNATIVES I CONSIDERED
LangGraph	Explicit graph with conditional and parallel edges, typed state, per-request checkpoints; easy to replay	A single prompt chain (not inspectable); a custom loop (reinvents checkpoints)
DuckDB + pandas + SciPy	Exact, in-process analytics; Wilson intervals, χ^2 and z-tests	Model-computed statistics (not reproducible)
FAISS, MiniLM + LSA, fused by RRF	Local and free; semantic and lexical signals complement each other	A hosted vector store (unnecessary at this scale)
Qwen2.5-1.5B-Instruct, local (llama.cpp)	No key, no cost, data stays on the machine; good enough for closed-list classification	Hosted models (key blocked during development; see below)
FastAPI + React	Streaming (SSE) for live agent progress; a real analyst workspace	A CLI only (harder to show evidence and charts)
SQLite	Sessions, checkpoints, telemetry, audit in one file	A server database (no benefit for a single-node prototype)

Dependencies, keys and cost

- **API keys:** none required. Nothing secret is hardcoded; all settings are environment variables (`.env.example`).
- **External services:** a one-time download from Hugging Face (MiniLM weights, and the local model if chosen). After that, no data leaves the machine.
- **Cost:** $0. When the local model is used it makes at most two calls per question; intent classification measured about 660 prompt tokens.
- **Latency:** deterministic answers p50 125 ms, p95 540 ms. With the local model, a warm routing call takes about a second; the first call after start-up took 8.7 s on an idle CPU.

Why a local model instead of a hosted API

The brief allows free, commercial or open-source hosted models. I deliberately chose a small open-source model that runs locally, for these reasons:

- **The hosted key I had did not work.** The API key available during development returned `401 API_KEY_SERVICE_BLOCKED`. I did not want the prototype, or its evaluation, to depend on a credential I could not rely on.
- **The model's job is narrow.** By design it never computes numbers or writes SQL. It only picks a question category from a closed list when the rules are unsure (6.9% of test questions) and phrases explanations of verified results. A 1.5B model constrained by a JSON grammar does this well: routing accuracy rises from 90.4% without it to 95.9% with it.
- **Anyone can run it.** Reviewers need no account, no API key and no budget. The whole system, model included, costs $0 to run.
- **The data stays on the machine.** Customer conversations are never sent to a third party, which matters for contact-centre data.
- **Results are reproducible.** The model file is pinned, so the benchmarks score the same model every time. Hosted models can change behind the same name, and rate limits or outages would have made the evaluation harder to repeat.

- **Answers never wait on a network.** If the model is missing or slow, the request falls back to the deterministic answer.

The trade-off. A local CPU model is slower and writes plainer explanations than a large hosted model: about 1 s per routing call, and tens of seconds for a model-written explanation. A hosted model can be added later behind the same router without redesigning anything (see §12).

> MY THINKING
>
> *When the key failed, instead of waiting I asked what the product actually needs a model for. The honest answer was: routing the questions the rules cannot, and phrasing explanations. Once I saw how narrow that job was, a small local model was not a compromise. It made the system cheaper, private by default, and easier to evaluate.*

6 · RESULTS

The four questions, answered

Q1 · Top 5 topics associated with negative sentiment in the last month

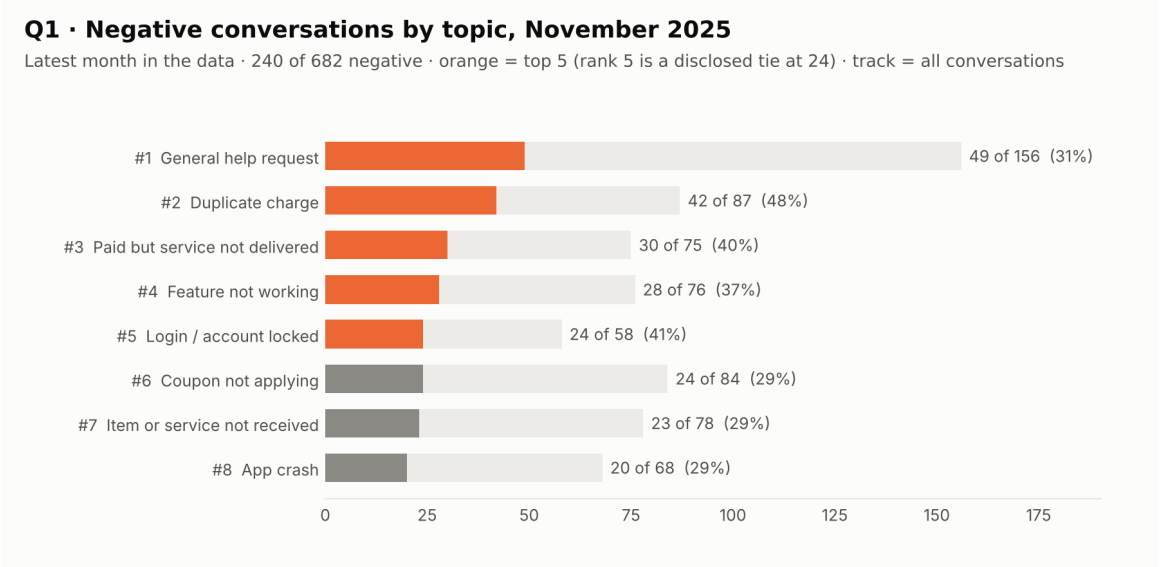


Figure 4. November 2025: 240 of 682 conversations negative. Top 5 highlighted; rank 5 is a tie.

RANK	TOPIC	NEGATIVE	OF	RATE
1	General help request	49	156	31.4%
2	Duplicate charge	42	87	48.3%
3	Paid but service not delivered	30	75	40.0%
4	Feature not working	28	76	36.8%
5	Login / account locked (tied with Coupon not applying, 24 of 84)	24	58	41.4%

The answer also states that Duplicate charge leads by rate, and that differences between topics are not statistically significant (χ^2 $p = 0.07$). The result matches an independent oracle computed from the raw CSV exactly.

Q2 · Agents who may benefit from empathy or communication coaching

The data has 2,987 agent IDs for 3,000 conversations, at most two conversations each. Any ranking of individuals would rest on one or two transcripts, so the answer says this plainly and gives what the data can support:

- **Workforce patterns:** only 3,215 of 10,966 negative customer turns (29.3%) received an acknowledgement or apology, and 1,340 of 2,338 assessable closing actions (57.3%) did not fit the issue.
- **A review queue of conversations** for team leads, each with its evidence.
- **Safeguards:** a minimum-sample gate (3 conversations) for any per-agent view, a human-review flag on every coaching answer, and refusals for personnel decisions.

> **MY THINKING**
>
> *This was the answer I thought hardest about. The brief's question invites a league table of agents, and I could have produced one. I would rather tell a team lead "the data cannot support this" than hand them a ranking that could affect someone's job on the strength of two chats. Reframing to patterns and a review queue keeps the question useful and the answer defensible.*

Q3 · How customer sentiment changed over a selected conversation

Turn-by-turn customer sentiment is shown in turn order, with turning points and the agent message just before each. For C0000479 (Coupon not applying · Voucher), sentiment worsened from +1.0 at the first follow-up to −1.0 at the end, across five turning points; the problem resurfaced after the customer said it was fixed. A Conversation X-Ray links each finding (escalation, urgency, repeated concern, unacknowledged frustration) to the exact messages.

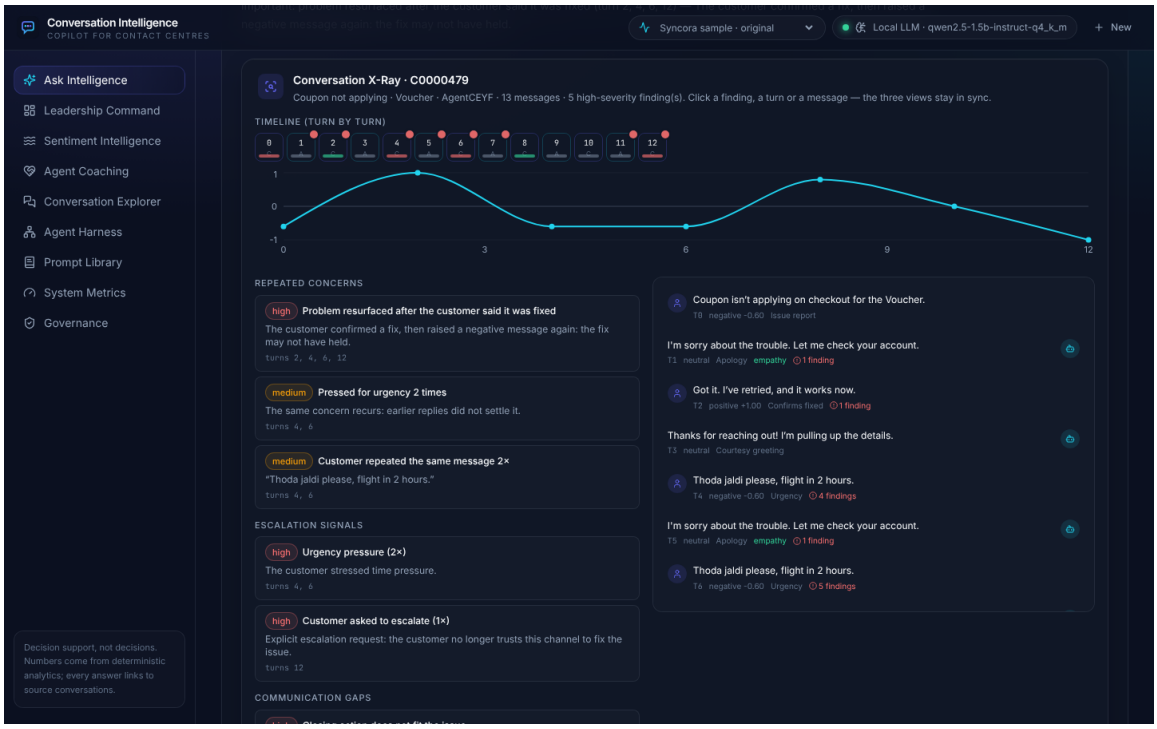


Figure 5. Conversation X-Ray: timeline, sentiment line, findings and transcript share one selection.

Q4 · Recurring concerns leadership should review first

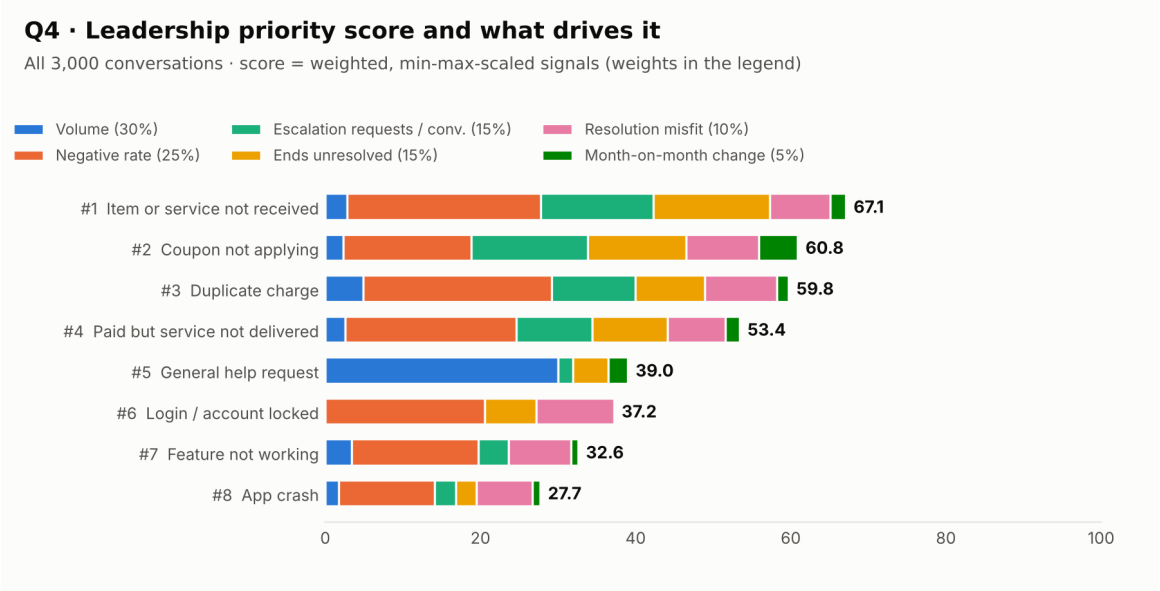


Figure 6. Leadership priority scores and what drives each one.

Item or service not received ranks first (67.1), then Coupon not applying (60.8) and Duplicate charge (59.8). The weights are shown with the chart, and a sensitivity analysis re-ranks under 13 weight scenarios: the top concern stays first in 92% of them. Each recommendation names the finding, its driver, an action, a decision owner, evidence and its uncertainty.

7 · BONUS EXTENSIONS

Retrieval, a specialised agent, and memory

1 · Knowledge retrieval

QUESTION FROM THE BRIEF	MY ANSWER
How is it indexed?	4-turn windows with an overlap of 2 (18,639 chunks), embedded twice (MiniLM and LSA) in exact-cosine FAISS indexes with metadata filters.
When is it triggered?	For similar-case questions; in parallel for coaching evidence; and after an analysis, to illustrate the top-ranked result with negative examples.
What evidence is returned?	One passage per conversation, with message IDs, CSV line references and each method's rank.
How does it improve the answer?	Analysts see real, on-topic transcripts beside every computed result, and can open the full conversation. Retrieval never produces a number.

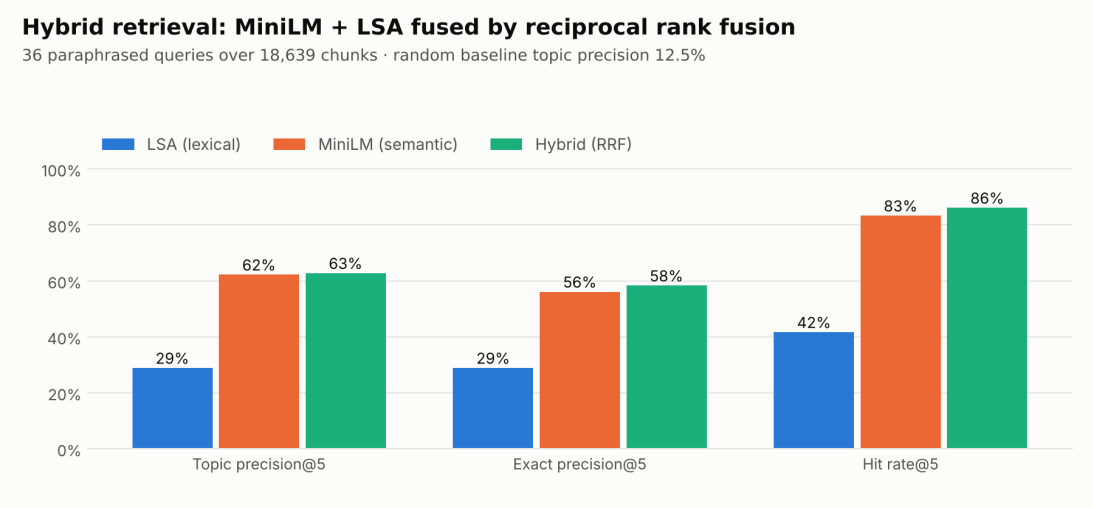


Figure 7. 36 paraphrased queries: hybrid hit@5 86.1% and topic precision@5 62.8% (MiniLM alone 83.3% / 62.2%).

The hybrid's gain over MiniLM alone is small, and the fusion weight was tuned on the same queries. I report both facts.

2 · Specialised LLM agent: a constrained local intent classifier

- **What:** Qwen2.5-1.5B-Instruct answers with exactly one category from a closed list, enforced by a JSON-schema grammar. Entities are validated before any query runs.
- **Why this approach:** classifying a question is a safe job for a small model, because every entity and number still comes from deterministic code. Fine-tuning was not justified by 426 templates and a few hundred benchmark questions.
- **How it fits:** it is the last automatic stage of the routing cascade, called for 6.9% of test questions.
- **How usefulness is evaluated:** a 73-question test split written before the router changed, frozen by SHA-256 and scored once. The router scores 95.9% with the model and 90.4% without it.

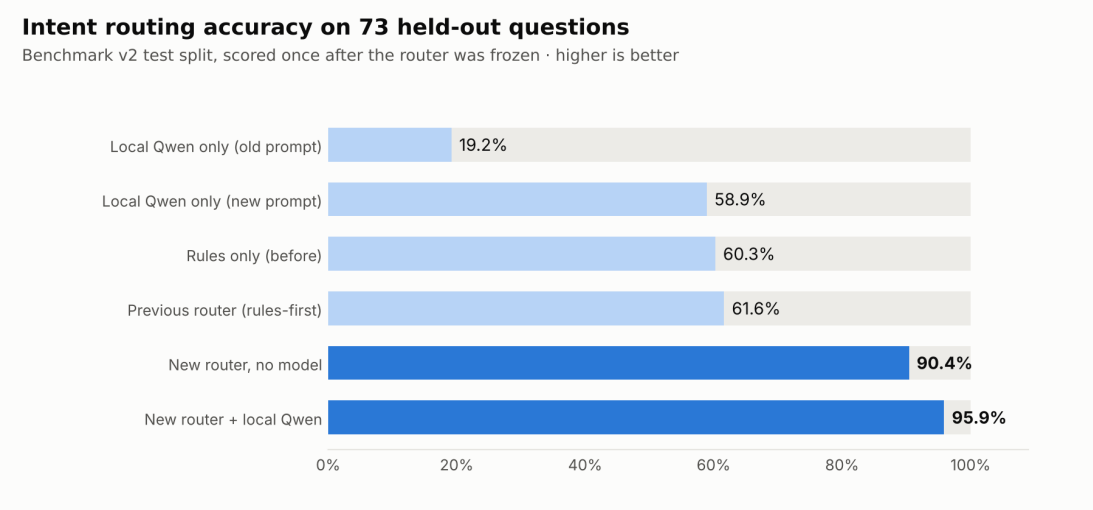


Figure 8. Routing accuracy on the held-out split, from the first prompt (19.2%) to the final router (95.9%).

MY THINKING

The model alone scored 19.2% with my first prompt and 58.9% after a careful rewrite. I could have kept tuning prompts, but each tweak was a guess I could not verify without spending the test set. I stopped asking the model to do the whole job and built the confidence cascade instead. I also built a semantic-similarity stage, calibrated it, and left it switched off, because no threshold reached 95% precision on validation.

3 · Memory integration

QUESTION FROM THE BRIEF	MY ANSWER
What is stored?	Per turn: question, intent, scope, plan, and deterministic references (ranked keys, conversation IDs). Never model prose.
How is it retrieved?	By session ID at the start of each request; references such as "the top two" or "the previous month" are resolved from it.
How does it affect later answers?	Follow-ups re-run the stored plan with the new scope, so "compare the top two" compares the real #1 and #2. 11 of 11 evaluation follow-ups routed correctly.
Reliability and privacy	References are re-validated against the database before reuse. Sessions are isolated, retained for 30 days (configurable) and deletable per session.

8 · BEYOND THE BRIEF

Six enhancements

After the four questions and the three extensions worked, I added six enhancements. Each one came from asking what an analyst would need in order to trust an answer, or to act on it. They follow the same rule as the rest of the system: they read verified results and never produce numbers of their own.

ENHANCEMENT	WHAT IT ADDS	WHY I BUILT IT
Zero-LLM Intelligence Router	Decides before any model call whether a model is needed. Analysts can also choose Analytics only or Hybrid AI.	Fact questions should cost nothing and never depend on a model. All four brief questions use zero model calls.
Query-driven dashboards	Turns each answer into a dashboard with a reason for each chart, filter chips and refinements that re-run the analysis.	An answer is usually the start of an investigation, not the end of one.
Conversation X-Ray	Escalations, repeated concerns and communication gaps in one conversation, each linked to the exact messages.	Q3 asks how sentiment changed; a team lead also needs to see where and why.
Root Cause Investigator	Tests factors associated with negative sentiment, with intervals, separating symptoms from candidate drivers.	To answer why without claiming cause from observational data.
AI Decision Replay	Step-through playback of routing, every node, each DuckDB operation, retrieval and model call, with secrets redacted.	Reviewers and analysts can check how an answer was produced.
Evidence Challenger	Re-reads every quote, discloses small samples, states counter-evidence and rewrites causal wording.	A second, adversarial check before anything is shown.

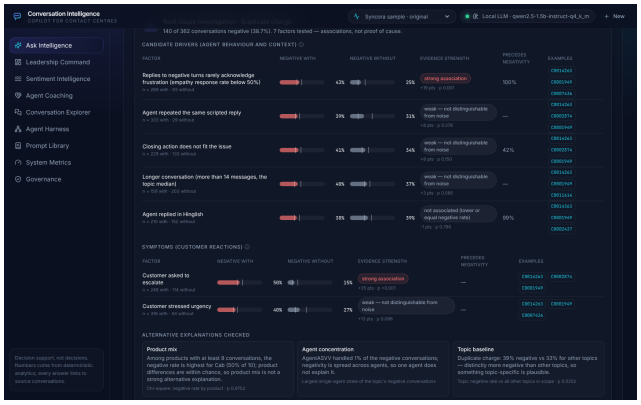


Figure 9. Root Cause Investigator: drivers vs symptoms, with intervals.

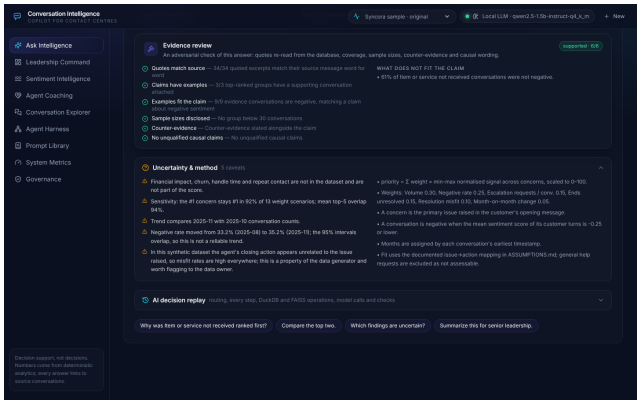


Figure 10. Evidence Challenger: what was checked, and what does not fit the claim.

MY THINKING

Building the Evidence Challenger exposed two real bugs: QA had been reporting zero resolved evidence items because one node overwrote another's report, and a repeated-phrase detector always returned zero. It also caught one of my own sentences: it rewrote "because of" into "associated with" in the tie disclosure for Q1. My wording was the problem.

9 · EVALUATION

What I measured, what worked, and what failed

I evaluated the system the way the brief suggests: task-specific metrics, representative successes and failures, grounding quality, and latency and consistency. Every benchmark was written before the change it measures.

Task-specific metrics

MEASURE	RESULT	HOW
Q1 vs independent oracle	Exact match	Separate code over the raw CSV, exact fractions
Intent routing (73 held-out)	95.9% · macro F1 0.932	Frozen split, scored once; 90.4% without a model
Refusals of unsupported questions	6 / 6 end to end	Revenue, CSAT, personnel decisions, off-topic
Follow-up routing	11 / 11	Multi-turn evaluation sessions
End-to-end QA pass rate	88% of 51 requests	All critical checks pass; the rest are Challenger warnings
Retrieval	hit@5 86.1% · topic P@5 62.8%	36 paraphrased queries; MiniLM alone 83.3% / 62.2%
Label agreement	78% accuracy · Cohen's κ 0.67	Template labels vs an independent lexicon
Automated tests	138 backend · 8 Vitest · 17 Playwright	All passing

Representative successes

- All four brief questions answered with zero model calls, correct counts, and evidence that opens the source transcript.
- "How much revenue did we lose to coupon issues?" is refused with a reason: the dataset has no revenue.
- "Compare the top two" after Q1 compares General help request and Duplicate charge, keeping November 2025.

Representative failures (kept visible, not fixed on the test split)

QUESTION	EXPECTED	GOT
Compare the sentiment mix across languages	sentiment distribution	topic comparison
Is delivery or payment worse for sentiment?	topic comparison	negative topics (model)
Why is the top negative topic so negative?	root cause	explain previous answer

A failure that taught me the most

While auditing my own submission, one Q4 score was off by 0.5. The cause was floating point: a conversation whose customer scores average exactly −0.25 on paper (for example −0.6, −0.6, −0.6, −0.6, +0.8, −0.6, +0.8, −0.6) sums to −0.24999999999999994 in floating point, so it was labelled not negative. Fixing it moved a priority score from 52.9 to 53.4 and the November count from 239 to 240. My independent checker had made the same mistake, because it also averaged with floats. The oracle now uses exact fractions, and a regression test pins the case.

> MY THINKING
>
> *Two implementations that share an assumption are really one implementation. Since then I check the checker: the oracle uses a different arithmetic from the system it verifies.*

Grounding quality

- **Excerpt fidelity 100%:** every quoted excerpt is re-read from DuckDB and must match word for word (34 of 34 in the Q4 example).
- **Grounded explanations 100%:** an explanation containing any number not in the verified result is rejected and replaced by the deterministic narrative.
- **Evidence coverage 92.3%:** the share of top-ranked claims with an attached example; missing examples are flagged as warnings.
- **Evidence relevance:** examples must fit the claim (negative examples for a negative claim); counter-evidence is stated alongside.

Latency and consistency

- **Deterministic answers:** p50 125 ms, p95 540 ms end to end on a local CPU. Retrieval alone: p50 39 ms.
- **With the local model:** about 1 s per warm routing call; the first call after start-up took 8.7 s on an idle CPU and 30–55 s under CPU contention. Model-written explanations take tens of seconds.
- **Consistency:** the deterministic path returns identical answers for identical questions. The routing benchmark records the stage that decided each question, so any change in behaviour can be traced to a rule, a threshold or the model.

10 · JUDGEMENT

What I did, what I chose not to do, and how I worked

I CHOSE NOT TO…	BECAUSE…
Rank individual agents	No agent has more than two conversations; a ranking would be noise with consequences.
Let a model write SQL or produce numbers	Numbers must be reproducible and testable; the registry keeps claims finite and reviewable.
Claim causes	Associations are tested with intervals and phrased as associations; customer reactions are treated as symptoms.
Fine-tune a model	426 templates and a small benchmark do not justify it; a constrained small model plus rules does the job.
Enable the semantic routing stage	It did not reach 95% precision on validation, so its own calibration keeps it off.
Tune on the test split	It would make the 95.9% meaningless; the three remaining errors stay listed.
Append data or keep demo data in the default build	Results must describe the supplied 3,000 conversations only.
Build authentication or multi-tenancy	Out of scope for a prototype; listed under next improvements.

How I worked

- **Data first.** Profiling, quality checks and the label mapping came before any agent, because every later answer depends on them.
- **Analytics core and registry next**, each operation tested against hand-counted fixtures.
- **Then the workflow:** LangGraph nodes, retrieval, memory and the API, followed by the analyst workspace.
- **Evaluation alongside, not after:** an oracle for Q1, a routing benchmark, a retrieval benchmark, end-to-end QA.
- **Routing optimisation as a controlled experiment:** baseline diagnosis, then rules, semantic stage, prompt and confidence router, then one score on the frozen test split.
- **Finally, an audit of my own submission** against every sentence of the brief, which found the float bug, a stale report and missing documentation. I fixed them and recorded the fixes.

11 · LIMITATIONS

What a reviewer should know

- **Labels are a judgement** over templated synthetic text; agreement with human labellers is not yet measured.
- **The routing benchmark is small and same-author:** 73 test questions written by the person who wrote the rules. The 95.9% is indicative, not precise.
- **The hybrid retrieval gain is small**, and its weight was tuned on the evaluation queries.
- **The local model is slow on CPU** for long explanations (tens of seconds), so interpretive answers in Hybrid mode are slower.
- **Root-cause signals come from template labels**, so associations may reflect how the data was generated.
- **Not production-hardened:** no authentication or role-based access, single-node storage, Docker images not verified end to end.

12 · NEXT IMPROVEMENTS

From prototype to production

AREA	NEXT STEP
Quality	Collect human labels in the review panel; measure agreement; re-write a routing benchmark with other authors.
Models	Optionally add a hosted model behind the same router: function calling over the operation registry, streamed and schema-constrained explanations, context caching, token budgets with fallback to the local model, and the same frozen test split as a gate.
Data platform	Managed warehouse and vector store; a job queue for labelling and re-indexing.
Security	SSO and role-based access, tenant isolation, a secret manager, retention policies per tenant.
Operations	OpenTelemetry tracing, CI/CD with evaluation gates, A/B tests of prompts and models on human-labelled sets, horizontal API scaling.

13 · REPOSITORY

What is in the repository, and how to run it

https://github.com/florina787/Data-sets/tree/claude/happy-edison-gef7ay/conversation-intelligence-copilot

Source code and setup/run instructions

`backend/` holds the FastAPI app, agents, analytics, retrieval, memory and tests. `frontend/` holds the React workspace and Playwright tests. Python 3.12+ with uv, Node 22+ and make; no API key.

```
git clone https://github.com/florina787/Data-sets.git
cd Data-sets/conversation-intelligence-copilot && git checkout claude/happy-edison-gef7ay
cp .env.example .env
make install && make prepare && make index
make dev # UI http://localhost:5173 · API docs http://localhost:8000/docs
make test && make e2e # 138 + 8 + 17 tests
make install-local-llm && make llm-download # optional local model
```

The README also covers Docker, VS Code launch configurations, PyCharm and a two-terminal setup.

Sample requests and outputs

`examples/requests/` holds six requests and `examples/outputs/` the matching responses as JSON and readable Markdown, generated from real runs: the four brief questions (with follow-ups), a retrieval request and an unsupported question. For example:

```
curl -s localhost:8000/api/chat -H 'Content-Type: application/json' \
  -d '{"question":"Which recurring customer concerns should contact-centre leadership review
first?"}'
```

Obtaining or preparing the dataset

The supplied sample is committed at `data/raw/cs_conversations.csv.zip`, so nothing needs to be downloaded. `make prepare` validates, cleans, labels and loads it into DuckDB (about 30 seconds), and `make index` builds the retrieval index. The data dictionary is in `docs/dataset/DATASET.md` and every interpretation in `ASSUMPTIONS.md`.

Environment variables and external services

VARIABLE	DEFAULT	PURPOSE
LLM_PROVIDER	auto	auto uses the local model if downloaded, otherwise none; none forces deterministic answers
LOCAL_MODEL_REPO / FILE / DIR	Qwen2.5-1.5B GGUF, data/models	Local model source and location
RETRIEVAL_MODE, RRF_K, HYBRID_LSA_WEIGHT	hybrid, 60, 0.15	Retrieval method and fusion settings
ENABLE_SYNTHETIC_DEMO	false	Demo conversations are loaded only when enabled
SESSION_RETENTION_DAYS	30	Memory retention

All settings are environment variables listed in `.env.example`; nothing secret is hardcoded and no API key is required. The only external service is a one-time download from Hugging Face (MiniLM weights, and the local model if chosen). Cost: $0.

> **A CLOSING NOTE**
>
> *The brief says you are not looking for a polished enterprise product, but for practical engineering judgement. I tried to spend my effort where judgement shows: deciding what the model should not do, refusing to overstate thin data, and measuring the system honestly, including the parts that are not yet good enough.*